



UNIVERSITÄT
LEIPZIG

Word Embeddings and Tokenisation

Natural Language Processing

Leonie Weissweiler

RECAP

ANNOTATION QUESTIONS

Annotation is the process of adding interpretative, linguistic information to a corpus.



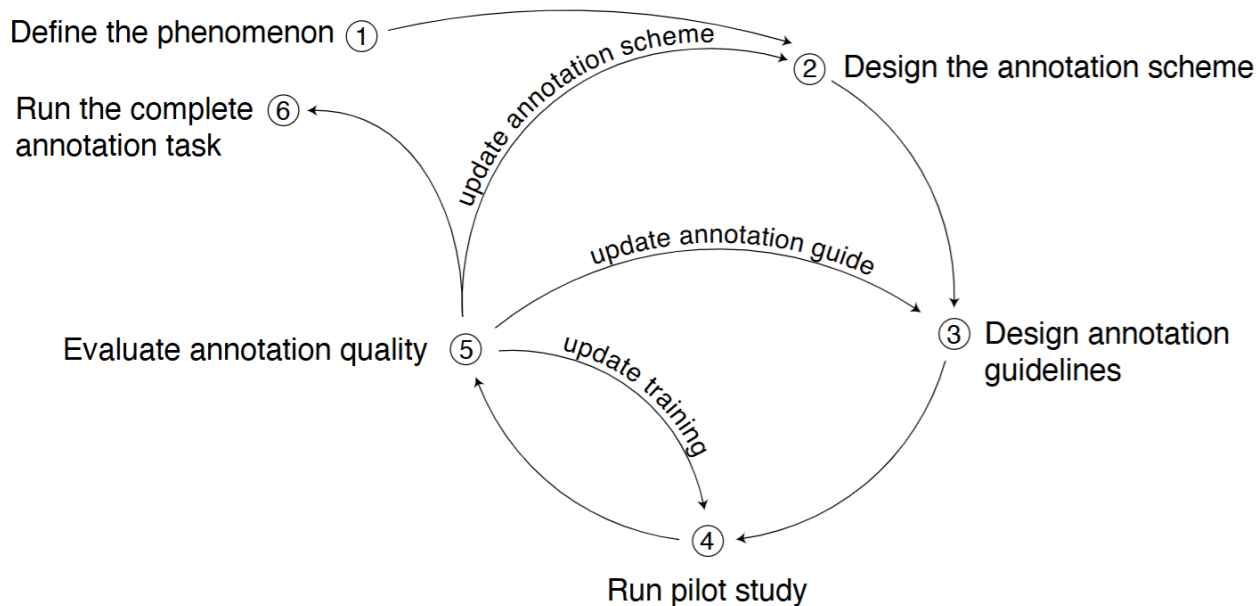
Questions:

Topic: "Google revenues" **Genre:** "News article"

- How can we encode annotations in a corpus?
- How can we collect the annotations?
- How can we ensure annotation quality?

ANNOTATION TASK DESIGN

Designing annotation tasks is an iterative process, similarly to software development.



CHANCE AGREEMENT

Let S be a sample of n objects, C a set of k classes, and γ_1 and γ_2 two independent annotators.

The observed agreement is the ratio of objects to which γ_1 and γ_2 assign the same class:

$$a_{\text{obs}} = \frac{|\{o \mid o \in S : \gamma_1(o) = \gamma_2(o)\}|}{n}.$$

Expected agreement. Probability that annotators γ_1 and γ_2 assign the same class:

$$a_{\text{exp}} = \sum_{c \in C} P(\mathbf{C}_{\gamma_1} = c) \cdot P(\mathbf{C}_{\gamma_2} = c)$$

COHEN'S K

Chance-corrected agreement coefficient:

$$\kappa = \frac{a_{\text{obs}} - a_{\text{exp}}}{1 - a_{\text{exp}}} = \frac{\text{observed agreement above chance}}{\text{potential agreement above chance}}.$$

Range of κ :

- $\kappa = 1$ indicates perfect agreement.
- $\kappa = 0$ indicates random agreement.
- $\kappa < 0$ indicates systematic disagreement.
- The lower bound for κ depends on the specific annotation scenario and approaches -1

MAXIMUM LIKELIHOOD LANGUAGE MODELS

A language model can be estimated via maximum likelihood estimation (MLE).

$$p_{\text{MLE}}(x_n \mid x_{<n}) = \frac{C(x_{<n}, x_n)}{\sum_{x_i \in V} C(x_{<n}, x_i)} = \frac{C(x_{<n}, x_n)}{C(x_{<n})}$$

the quick brown fox jumps over the lazy dog			
$x_{<n}, x_n$	$C(x_{<n})$	$C(x_{<n}, x_n)$	$p(x_n \mid x_{<n})$
the quick	2	1	1/2 = 0.5
the quick brown	1	1	1/1 = 1
the quick brown fox	1	1	1/1 = 1

Problems?

- We cannot estimate unobserved sequences, independently of their probability
- Most sequences with $n > 5$ are never observed. → Zipf's Law

AUTO-REGRESSION

Model the probability of a sentence as a series of next-word predictions.

1. Model sentence probability with conditional probabilities: multiplication rule

$$p(x_1, \dots, x_n) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_1, x_2) \cdot \dots \cdot p(x_n|x_{<n}) = \prod_{k=1}^n p(x_k|x_{<k})$$

2. Markov assumption: Limit the condition of each probability to length τ :

$p(x_k|x_{<k}) \approx p(x_k|x_{k-\tau}, \dots, x_{k-1})$. This example uses $\tau = 1$

$p(x_1, \dots, x_n) = p(\text{the, quick, brown, fox, jumps, over, the, lazy, dog})$

= $p(\text{the} | < s >)$

· $p(\text{quick} | \text{the})$

· $p(\text{brown} | \text{quick})$

· ...

· $p(\text{dog} | \text{lazy})$

SMOOTHING: ADD-ONE (LAPLACE)

Add 1 to the count of every n-gram before the likelihood estimation to avoid zero-probabilities.

$$p_{\text{add-1}}(x_n \mid x_{n-\tau}, \dots, x_{n-1}) = \frac{1 + \mathbf{C}(x_{n-\tau}, \dots, x_{n-1}, x_n)}{|\mathbf{V}| + \sum_{x_i \in \mathbf{V}} \mathbf{C}(x_{n-\tau}, \dots, x_{n-1}, x_i)}$$

3-gram counts

3-gram	Sam	house	eggs	ham	box
i am	2	0	0	0	0
in a	0	8	0	0	7
like green	0	0	9	0	0

3-gram counts (+1)

3-gram	Sam	house	eggs	ham	box
i am	3	1	1	1	1
in a	1	9	1	1	8
like green	1	1	10	1	1

3-gram probabilities (MLE)

3-gram	Sam	house	eggs	ham	box
i am	1	0	0	0	0
in a	0	0.54	0	0	0.46
like green	0	0	1	0	0

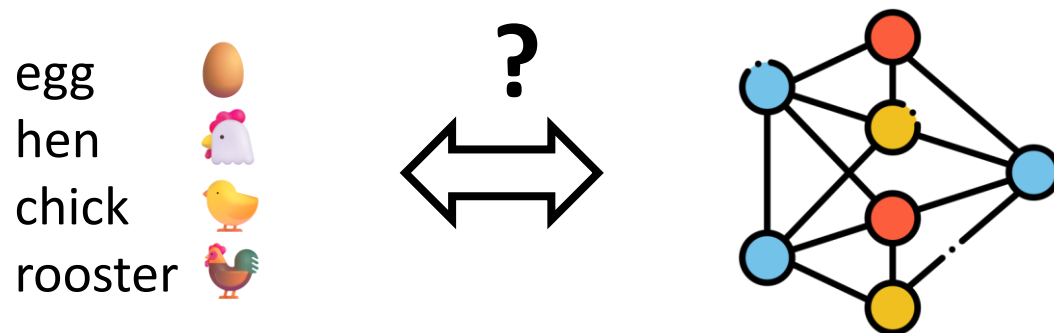
3-gram probabilities (add-1)

3-gram	Sam	house	eggs	ham	box
i am	0.43	0.14	0.14	0.14	0.14
in a	0.05	0.40	0.05	0.05	0.35
like green	0.07	0.07	0.71	0.07	0.07

QUESTIONS?

VECTORS FOR MEANING

WORDS AS INPUT TO LANGUAGE MODELS



REQUIREMENT: SIMILARITY

- egg != chick
- hen != rooster
- egg == egg

One-Hot vectors

egg = [1,0,0,0]

chick = [0,1,0,0]

hen = [0,0,1,0]

rooster = [0,0,0,1]

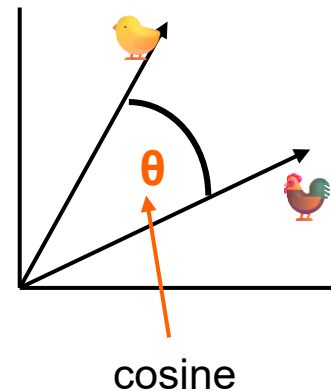
COSINE SIMILARITY

- Chick not more similar to hen than to rooster
- egg more similar to chick than to rooster
- Hen and rooster similar
- **→ cosine similarity**

egg = [1,0,0,0]

chick = [0,1,0,0]

$S_c(\text{egg}, \text{chick}) = 0$



$$S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}}$$

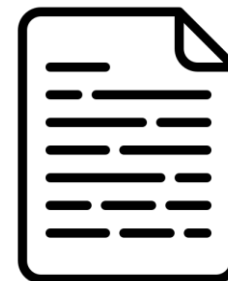
Scalar product of both vectors

Euclidian norms of both vectors

REQUIREMENT: AUTOMATIC GENERATION

—Solution: use a corpus

- The chicken lays an egg.
- A chick hatches from the egg.
- The chick grows into a hen or a rooster.



Distributional Semantics

You shall know a word by the company it keeps (Firth, J. R. 1957:11)

- → words in similar contexts have similar meaning

REMOVE PUNCTUATION

- The hen lays an egg
- A chick hatches from the egg
- The chick grows into a hen or a rooster

LOWERCASE

- the hen lays an egg
- a chick hatches from the egg
- the chick grows into a hen or a rooster

LEMMATISE

- the hen lay a egg
- a chick hatch from the egg
- the chick grow into a hen or a rooster

CREATE A VOCABULARY

- the hen lay a egg
- a chick hatch from the egg
- the chick grow into a hen or a rooster
- $V = \{a, chick, egg, from, grow, hatch, hen, into, lay, or, rooster, the\}$

CO-OCCURRENCE MATRIX

	a	chick	egg	from	grow	hatch	hen	into	lay	or	rooster	the
egg	2	1		1		1	1		1			2
chick	3		1	1	1	1	1	1		1	1	2
hen	3	1	1		1			1	1	1	1	2
rooster	2	1			1		1	1		1	1	1

–egg = [2,1,0,1,0,1,1,0,1,0,0,2]

–chick = [3,0,1,1,1,1,1,1,0,1,1,2]

–hen = [3,1,1,0,1,0,0,1,1,1,1,2]

–rooster = [2,1,0,0,1,0,1,1,0,1,1,1]

COSINE SIMILARITY USING THE NEW VECTORS

–egg = [2,1,0,1,0,1,1,0,1,0,0,2]

–chick = [3,0,1,1,1,1,1,1,0,1,1,2]

–hen = [3,1,1,0,1,0,0,1,1,1,1,2]

–rooster = [2,1,0,0,1,0,1,1,0,1,1,1]

chick ↔ hen 0.878

hen ↔ rooster 0.876

chick ↔ rooster 0.855

egg ↔ chick 0.787

egg ↔ hen 0.744

egg ↔ rooster 0.669

- **Chick not more similar to hen than to rooster**
- **Egg more similar to chick than to hen**
- **Hen and rooster similar**

EFFICIENCY ISSUES

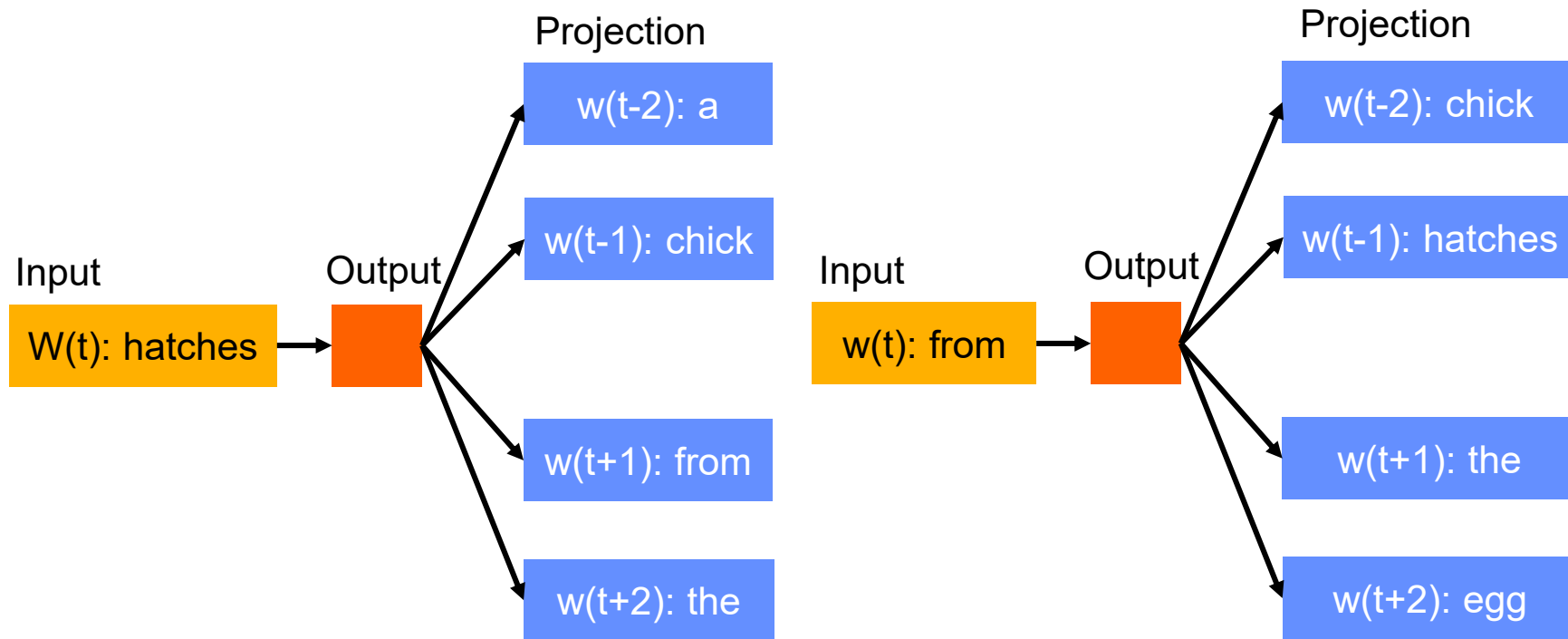
- **Zipf's Law**: most words are rare
- → very long word embeddings
- Problem 1: complexity of vector multiplication
- Problem 2: vectors with many zeroes
- → similarity no longer representative

$$p(n) \sim \frac{1}{n}$$

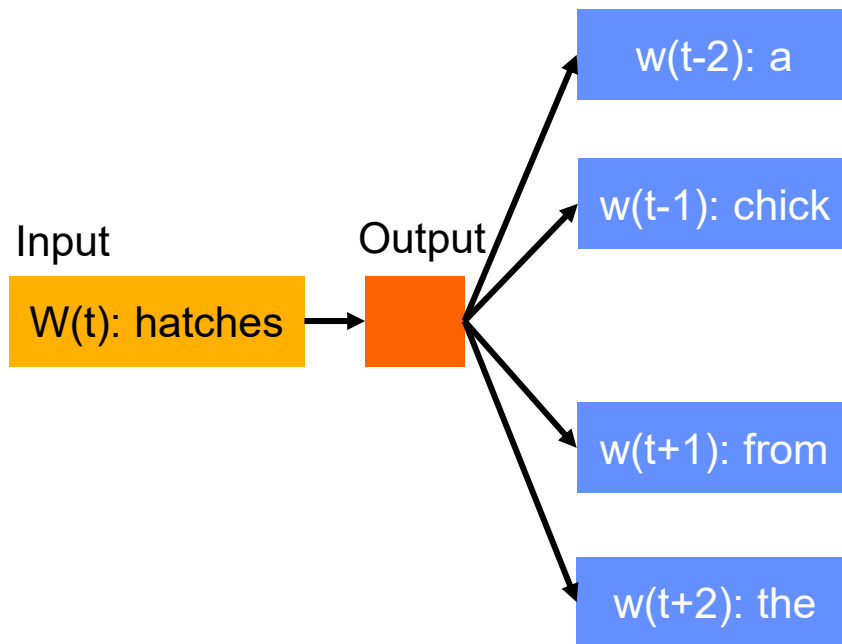
- **Goal** reduce dimensionality
- **Solution** have embeddings be the output of a language model, dimensionality can be whatever we choose

WORD2VEC: SKIPGRAM UND CBOW

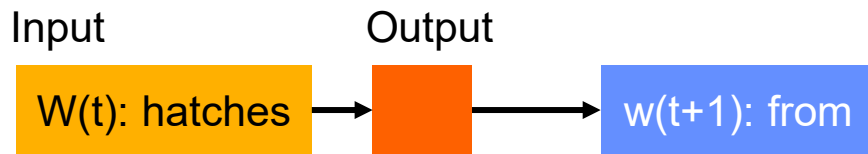
SKIP-GRAM: A CHICK HATCHES FROM THE EGG



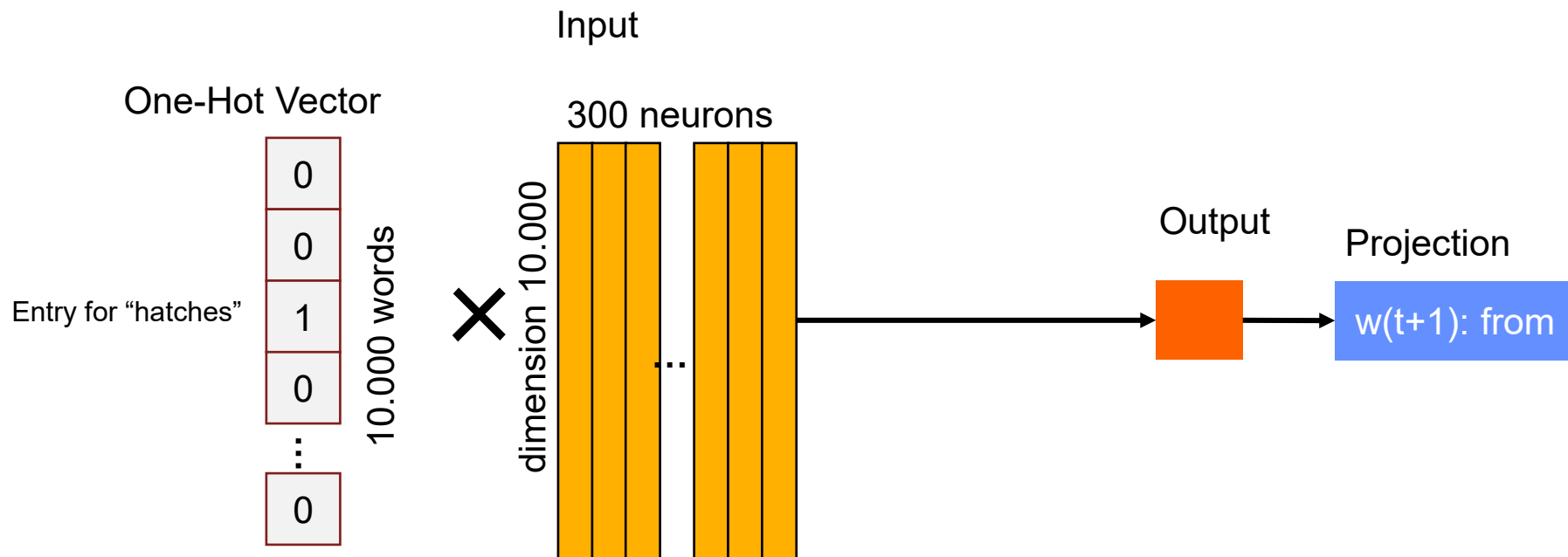
SKIP-GRAM



SKIP-GRAM

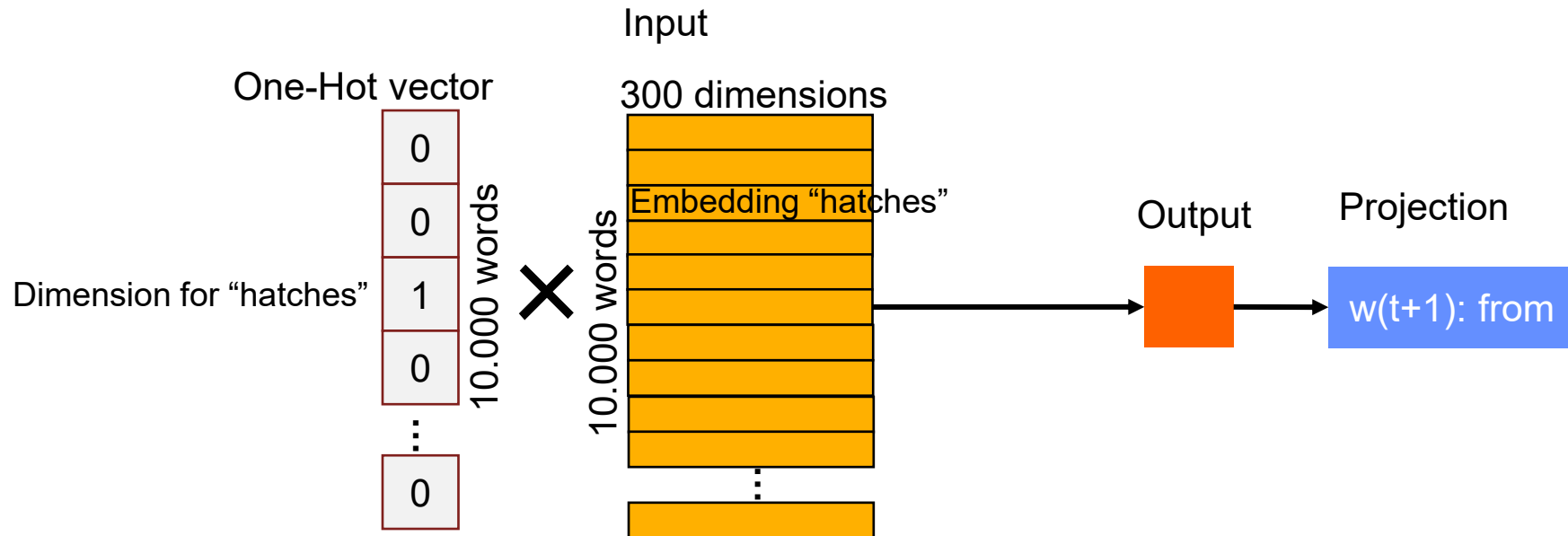


SKIP-GRAM



SKIP-GRAM

Word Embedding Matrix



SKIP-GRAM

Word Embedding

Input

300 dimensions

Embedding for “hatches”

Output

Projection

$w(t+1)$: from

SKIP-GRAM

Attention! Not the actual word embedding, just model-internal

Input

Word Embedding

300 dimensions



Embedding for
“hatches”



Output Embedding



Projection

$w(t+1)$: from

SKIP-GRAM

Word Embedding

Input

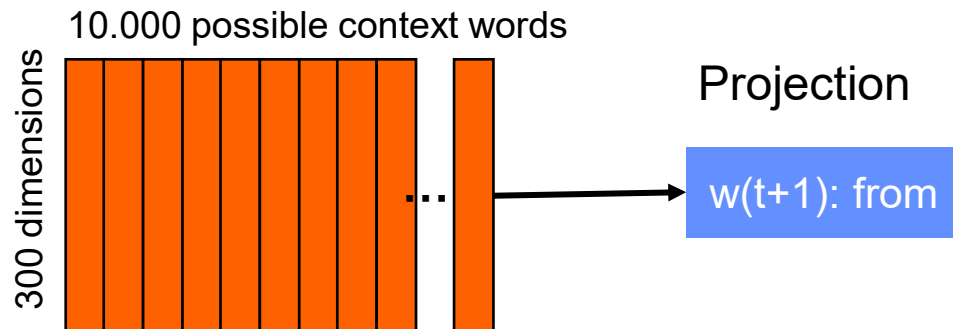
300 dimensions

Embedding for “hatches”



Attention! Not the actual word embedding, just model-internal

Output Embedding



SKIP-GRAM

Attention! Not the actual word embedding, just model-internal

Input

Word Embedding

300 dimensions

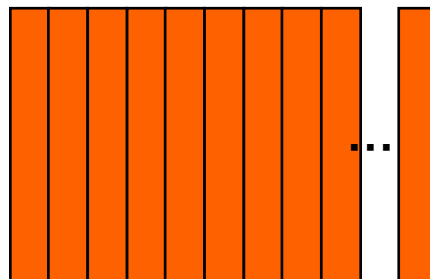
Embedding for "hatches"



Output Embedding

10.000 possible context words

300 dimensions



Softmax

Output probability distribution

10.000 possible context words



$p(\text{"from"}|\text{"hatches"})$

SKIP-GRAM

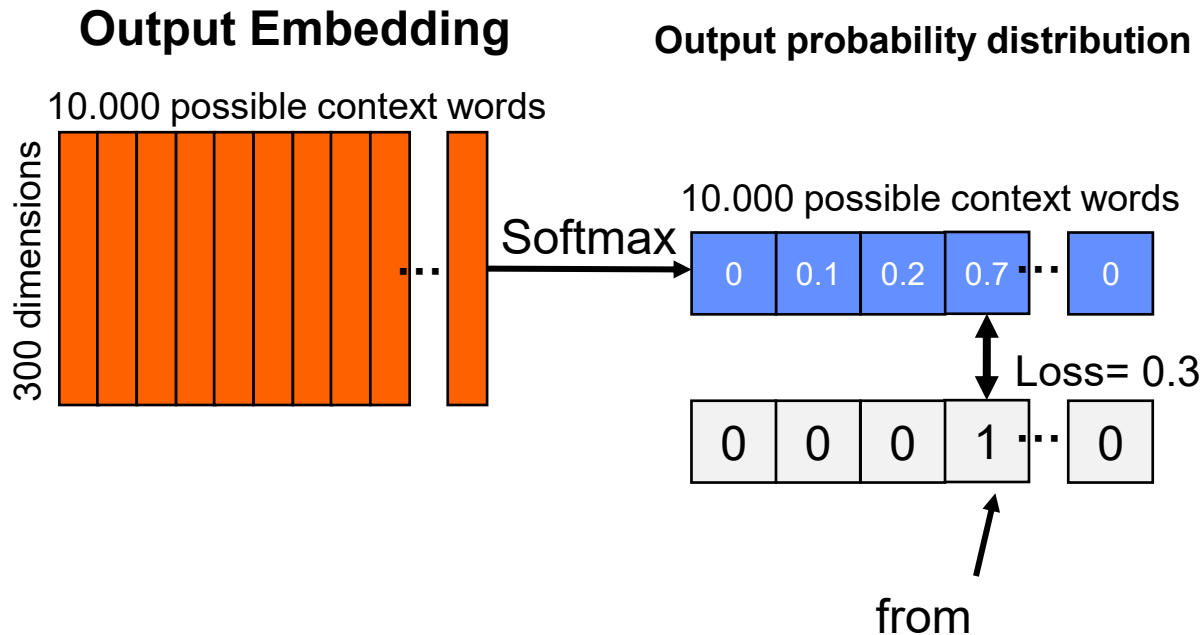
Attention! Not the actual word embedding, just model-internal

Input

Word Embedding

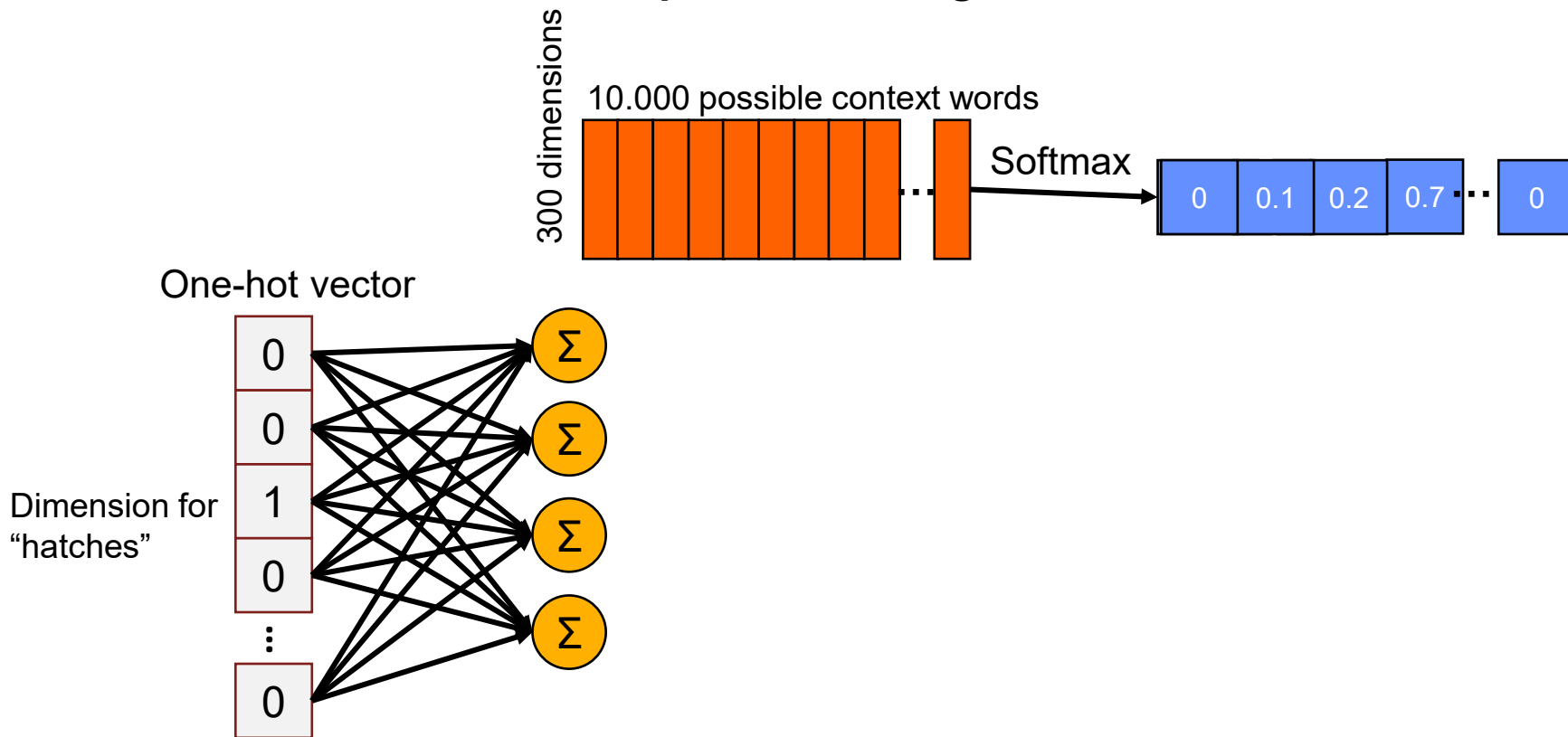
300 dimensions

Embedding for "hatches"

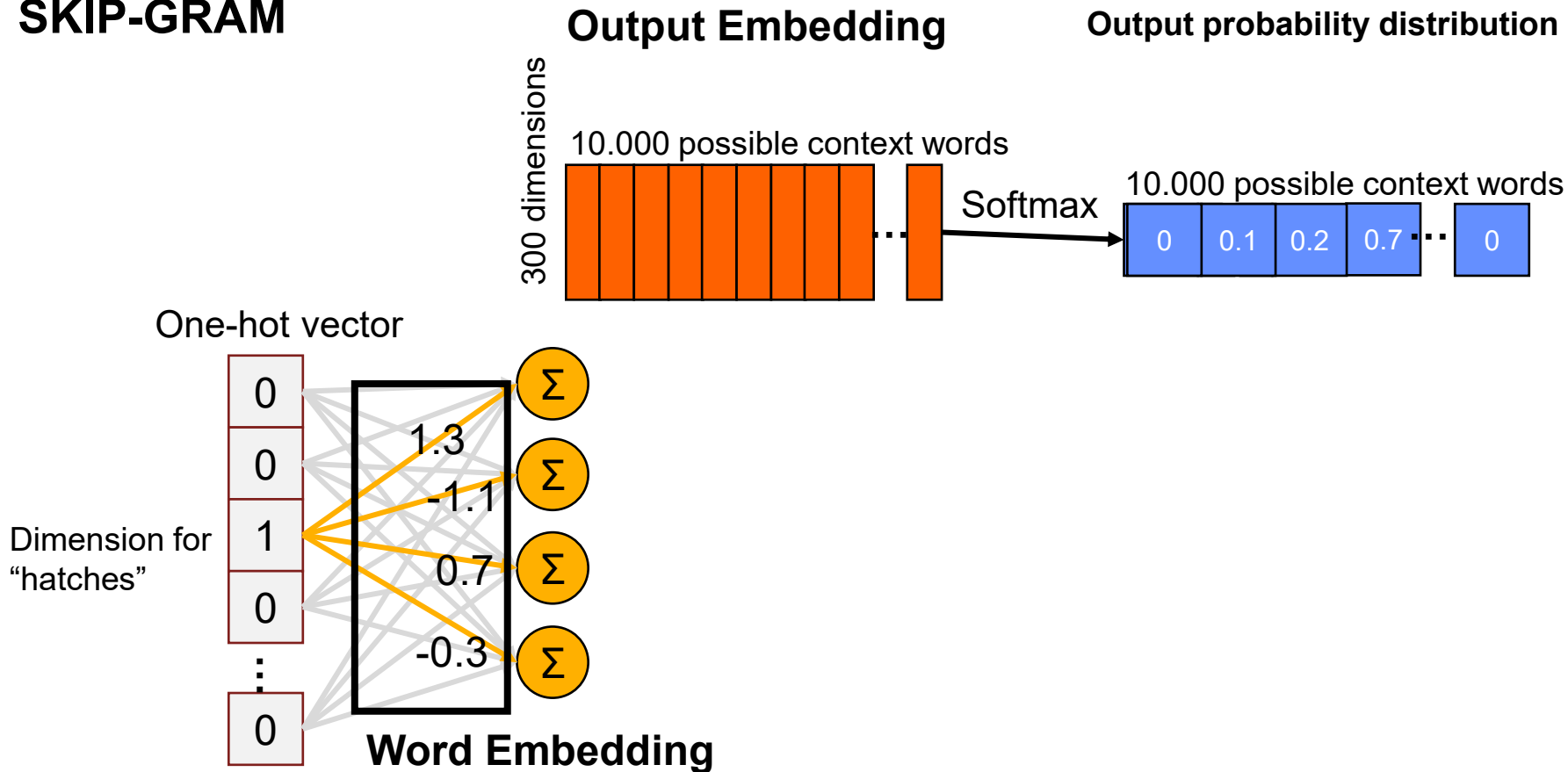


SKIP-GRAM

Output Embedding



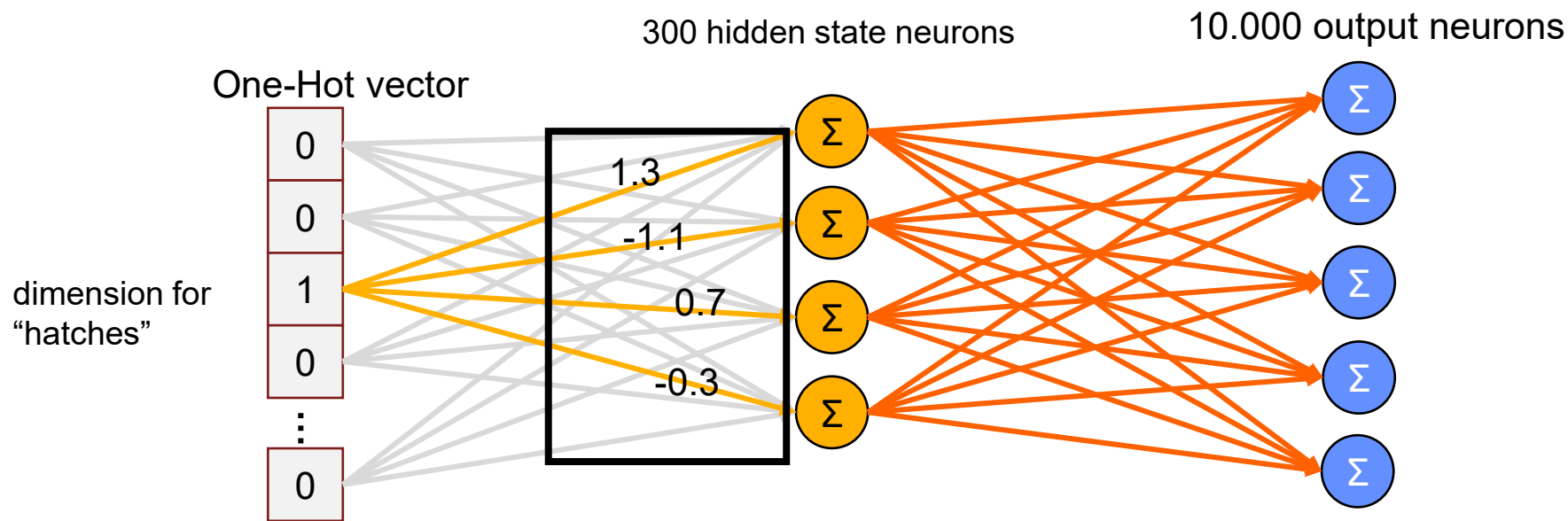
SKIP-GRAM



SKIP-GRAM

Word Embedding

Output Embedding



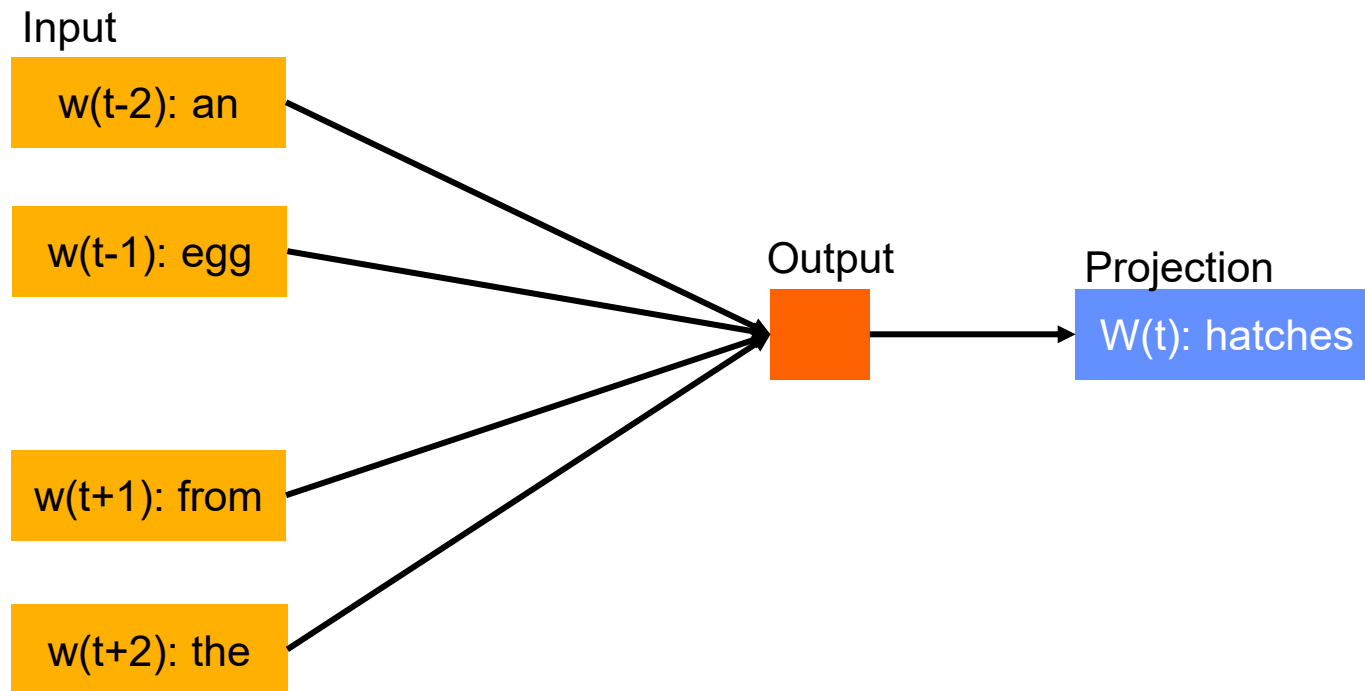
THE SKIPGRAM FORMULAS

$$\underbrace{\frac{1}{T} \sum_{t=1}^T}_{\text{Sum over all time steps (positions in the sentence)}} \underbrace{\sum_{-c \leq j \leq c, j \neq 0}}_{\text{Sum over all context words (c=2)}} \underbrace{\log p(w_{t+j} | w_t)}_{\text{Probability of the context word given the focus word}}$$

Scalar product of word embedding and context vector

$$p(w_O | w_I) = \frac{\exp \left(\underbrace{v'_{w_O}}_{\text{word embedding}}^\top \underbrace{v_{w_I}}_{\text{context vector}} \right)}{\sum_{w=1}^W \exp \left(\underbrace{v'_w}_{\text{word embedding}}^\top \underbrace{v_{w_I}}_{\text{context vector}} \right)} \Bigg\} \text{softmax}$$

CONTINUOUS BAG OF WORDS (CBOW)

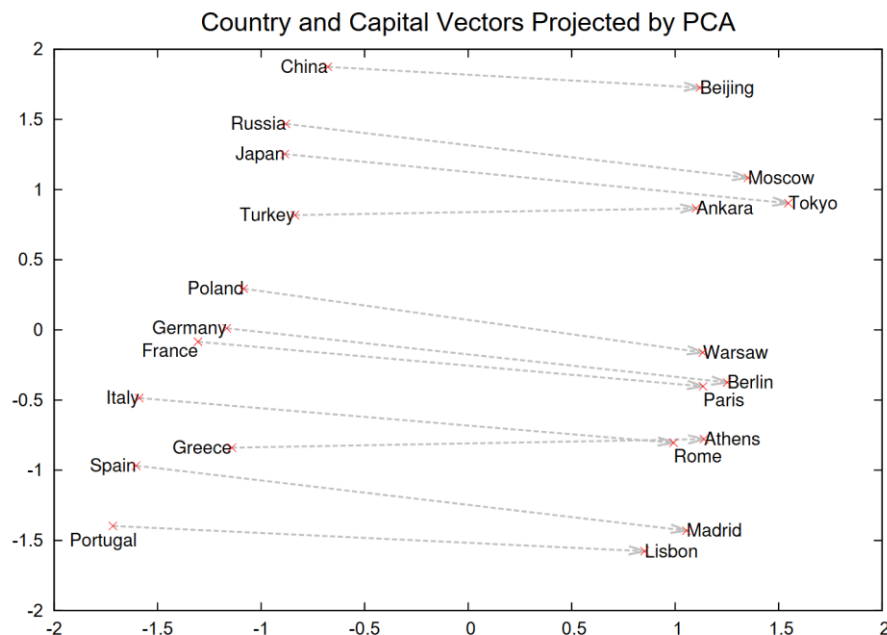


RESULTS AND EVALUATION

ANALOGICAL RELATIONSHIPS

Newspapers			
New York San Jose	New York Times San Jose Mercury News	Baltimore Cincinnati	Baltimore Sun Cincinnati Enquirer
NHL Teams			
Boston Phoenix	Boston Bruins Phoenix Coyotes	Montreal Nashville	Montreal Canadiens Nashville Predators
NBA Teams			
Detroit Oakland	Detroit Pistons Golden State Warriors	Toronto Memphis	Toronto Raptors Memphis Grizzlies
Airlines			
Austria Belgium	Austrian Airlines Brussels Airlines	Spain Greece	Spainair Aegean Airlines
Company executives			
Steve Ballmer Samuel J. Palmisano	Microsoft IBM	Larry Page Werner Vogels	Google Amazon

THE SEMANTIC EMBEDDING SPACE



RELATIONSHIPS BETWEEN PAIRS OF WORDS

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

QUESTIONS?

TOKENISATION

TOKENISATION

Given a text in the form of a character sequence, the task of tokenization is to split it into meaningful units called tokens.

“Friends, Romans, Countrymen, lend me your ears!”

“ Friends , Romans , Countrymen , lend me your ears ! ”

Terminology:

- A token is typically a word-like character sequence, or punctuation.
- A type relates to a token as a class relates to an object.

WHY IS TOKENISATION NECESSARY?

- Alphabetic and syllabic languages encode sounds as set of characters or short character sequences, reused to express an infinite amount mostly unrelated meaningful statements.
- Logographic and continuous script languages represent meaning as characters or short character sequences, but have no word boundaries, such as spaces.
- Corpus linguistics and statistical language processing requires sufficiently large unit (token) populations. Long units are too infrequent, and different units may have the same meaning.
- A tokenized text encodes knowledge of language in terms of how it is composed of basic units of meaning.

WHAT ARE MEANINGFUL UNITS OF TEXT?

- Character n-gram. $n \leq 3$ captures aspects of writing style.
- Syllable. Basic unit of speech uttered with a single effort of articulation.
- Morpheme. Minimal unit of meaning in language.
- Word. Sequence of one or more morphemes; an item of vocabulary.
- Phrase. A sequence of words expressing a notion (e.g., names, idioms, figures of speech).

LINGUISTIC TERMINOLOGY [OED]

- A syllable is defined as “a vocal sound or set of sounds uttered with a single effort of articulation and forming a word or an element of a word; [...] also, a character or set of characters forming a corresponding element of written language.”
- A morpheme is defined as “a minimal and indivisible morphological unit that cannot be analysed into smaller units (e.g. ‘in’, ‘come’, and ‘?ing’, forming ‘incoming’).”
- A word is defined as “an element or unit of speech [or] language”. “Any of the sequences of one or more sounds or morphemes (intuitively recognized by native speakers as) constituting the basic units of meaningful speech used in forming a sentence or utterance in a language (and in most writing systems normally separated by spaces); a lexical unit other than a phrase or affix; an item of vocabulary, a vocable.”
- A phrase is defined as “a small group or collocation of words expressing a single notion, or entering with some degree of unity into the structure of a sentence; a common or idiomatic expression.”

HEURISTIC TOKENISATION

- Morphemes would be the close-to-ideal target unit of meaning for tokenizing a text. Morpheme sequences capture text semantics in an almost lossless way.
- As a first approximation, traditional tokenization therefore targets words, separating them from other kinds of syntactical units, such as punctuation.
- As a second approximation, the word-like tokens are then normalized using heuristic morphology reduction techniques.
- Recently, in the wake of developing deep learning-based language models, sub-word tokenization approaches have emerged. These approaches approximate morphemes directly by analysing sub-word frequencies.

APPROACHES

- 1. Naïve heuristics:
 - Whitespace: A token is every character sequence separated by whitespace characters.
 - Unix tool: `tr -sc 'A-Za-z' '\n' < text.txtq`
 - TREC: A token is every alphanumeric sequence of characters of length > 3 , separated by a space or punctuation mark.
- 2. Linguistic / word tokenization
 - Manually construct a set of rules and apply them in order.
 - Each rule describes how to split a string into smaller tokens.
- 3. Learned / subword tokenization
 - Split tokens based on observed character sequence frequencies in a training corpus.

SPECIAL CASES

- Contractions: Apostrophes can be a part of a word, a part of a possessive, or just a mistake it's, o'donnell, can't, 80's, men's, master's degree, shriner's
- Hyphenated compounds: Hyphens may be part of a word or a separator. Some words refer to the same concept with or without hyphen. winston-salem, e-bay, wal-mart, active-x, far-reaching, 20-year-old
- Compounds: wheelchair, Computerlinguistik
- Other special characters: Special characters may form part of words. M*A*S*H, I.B.M., Ph.D., C++, C#, <http://www.example.com>.
- Numbers: Numbers form tokens of their own, and may contain punctuation as well: 6.5, 1e+010
- Phrases, named entities, phone numbers, dates: San Francisco, (800) 234-2333, Mar 11,1983.

REGULAR EXPRESSION TOKENISER

- `consonant="[bcdfghj-np-tvxz]",clitic="(?:'|:|-|'s|'d|'m|'ll|'re|'ve|n't)"`
- Insert space around punctuation which are unambiguous separators. `s/([?!()\\"\/\|])/ \1/g`
- Insert space around commas that aren't inside numbers. `s/(\D),/\1 , /g → s/,(\D)/ , \1/g`
- Insert space around single quotes not preceded by letter. `s/(\s')/\1 /g → s/(\W)'/\1 '/g`
- Insert space around unambiguous word-suffix clitics and punctuation. `s/(clitic\s)/ \1/g` and `s/(clitic)(\W)/ \1 \2/g`
- Split by whitespace (`/\s+/`) to obtain a list of tokens
- For each token, insert a space before full stops at the end of t, if t is not an abbreviation (i.e., t occurs in A, t is a letter–dot sequence, or t is an upper case letter followed by consonants and a dot) For each $t \in T$: If $t \in A$, $t \in L(/ \backslash w + \backslash . /)$, and $t \in L(/ ^ (([A-Za-z] \backslash .) \{ 2, \} | [A-Z][consonant] + \backslash .) \$ /)$ apply `s/ \backslash . \$ / \backslash . /`
- Return a whitespace-separated string

TOKEN NORMALISATION

Given a token (i.e., a word-like unit), map it to a “basic form” (e.g., its stem, lemma, or morpheme) to which semantically equivalent token variants (e.g., derivations or inflections) are also mapped.

Causes of token variability:

- Orthography. Surface-level variations in spelling and writing.
- Morphology. Variations of word formation due to inflection, derivation, and compounding.
- Lexical-semantic variation. Use of semantically same / similar words in different contexts.

Normalization problems:

- Undergeneralization. Not all equivalent tokens are mapped to the same canonical form.
- Overgeneralization. Non-equivalent tokens are mapped to the same canonical form.
- Language-dependency. Different rule-sets, dictionaries, and algorithms per language.

ORTHOGRAPHIC NORMALISATION

- Case folding. Lower-casing of all tokens. “Apple” → “apple”: Apple (fruit) at beginning of sentence vs. Apple (company).
- Unicode normalization. Mapping Unicode characters to their closest ASCII equivalent. “résumé” → “resume”: resume (curriculum vitae) vs. resume (continue).
- Character class reduction. Removal of non-alphanumeric characters. “C++” → “C” or “general-purpose” → “generalpurpose”: Names, acronyms, and compounds can be distorted.
- Spelling correction. Correction of mistakes and minor variations due to dialect. “devops” → “doves”: overcorrection of, e.g., names, memes, or technical jargon.

WORD TOKENISATION

Properties:

- Top-down analysis of surface text structure.
- Heuristic operationalization of a linguist's morphological analysis.
- Results in human-interpretable tokens.
- Language-specific knowledge and resources are required: rule sets and dictionaries.
- The vocabulary of tokens grows according to Heap's law.
- Token normalization is an error-prone process.

Applicability:

- Linguistic corpus construction
- Bag of words models and sparse vector representations

SUBWORD TOKENISATION

Idea: Learn tokenization rules based on frequently occurring character sequences in a corpus

BYTE-PAIR ENCODING (BPE)

Byte-Pair Encoding: [Gage 1994]

- “The algorithm compresses data by finding the most frequently occurring pairs of adjacent bytes in the data and replacing all instances of the pair with a byte that was not in the original data. The algorithm repeats this process until no further compression is possible, either because there are no more frequently occurring pairs or there are no more unused bytes to represent pairs. The algorithm writes out the table of pair substitutions before the packed data.”

Learning tokenization rules: [Sennrich et al., 2015]

- Process token sequences of a corpus instead of byte sequences.
- Starting point: Every character in the alphabet is a token.
- Iteratively identify the most frequent adjacent token pair, and use them as merging rule.

REMARK

Byte-pair encoding was originally proposed as a data compression algorithm. Sennrich et al., 2015, adapted it as a heuristic for corpus-relative tokenization. The motivation to do so was to avoid out-of-vocabulary tokens in machine translation, which frequently cannot be translated. By falling back on subword tokens, similar to stems, lemmas, or morphemes, unknown words were represented by substrings, based on the intuition that translators often can form an idea of how to translate an unknown word based on substrings that happen to be similar to known words.

TRAINING BPE

Input: D. A set of sequences of characters over alphabet Σ ; the corpus of documents.

k. The maximal size of the vocabulary V , $|V| \leq k$.

Output: V . Set of tokens, where $t \in V \subseteq \Sigma^*$; the vocabulary.

R. Sequence of token pair replacement rules.

BPE_train(D, k)

$V = \Sigma$, $R = \emptyset$

$D' = \text{pre_tokenize}(D)$ // Sequence of word-like tokens in D as character sequences.

$I = \text{index}(D')$ // Dictionary that maps tokens t to their token frequency $t_f(t, D)$.

1. FOR $i = 1$ TO k DO

1. $t_L, t_R = \text{most_frequent_token_sequence}(V, D')$.
2. $R = R + (t_L, t_R)$
3. $V = V \cup \{t_L + t_R\}$
4. FOREACH $t \in I$ DO $t = \text{replace_all}((t_L, t_R), t_L + t_R, t)$

2. return(V, R)

USING BPE

Input: d. A sequence of characters $(c_1, \dots, c_n)$ over alphabet Σ , the document.

R. A sequence of token pair replacement “rules” $(r_1, \dots, r_m)$, where $r_i = (t_L, t_R)$
and $t_L, t_R \in V \subseteq \Sigma^*$.

Output: The document as token sequence $(t_1, t_2, \dots)$

BPE_tokenize(d, R)

- $d' = \text{pre_tokenize}(d)$ // Sequence of word-like tokens in d as character sequences.
- FOR $i = 1$ TO m DO
 - $(t_L, t_R) = r_i$
 - $d' = \text{replace_all}((t_L, t_R), t_L + t_R, d')$
- return(d')

REMARKS

- `replace_all(s1, s2, s)` replaces every occurrence of sequence s₁ with sequence s₂ in sequence s.
- `pre_tokenize(D)` returns the corpus as a sequence of “pre-tokens” D'.
 - Use of a (naïve) tokenizer for every d ∈ D and concatenation of the token sequences to obtain D'.

My kingdom for a horse! → My kingdom for a horse!

- Typically, a word boundary character c_{word} ∉ D is added as prefix to every token. For languages that indicate a word boundary with a space, the last space ahead of a token is replaced with c_{word}. Adding other boundary characters is a design choice (e.g., for newlines, new paragraphs, new sentences, etc.).

My kingdom for a horse! → My _kingdom _for _a _horse!

- Each token is split into a sequence of characters.

My _kingdom _for _a _horse! → M y _k i n g d o m _f o r _a _h o r s e!

MORE REMARKS

- Here, a “sequence of characters” refers to individual characters, instead of a string (character sequence). In practice, allocating and merging many immutable string objects is avoided. Instead of splitting and merging strings, the initial pre-token strings (word-like units) are doubled in length and a space character is added before each character. Upon each merging using `replace_all` of two adjacent tokens (initially characters), the space between the pair of tokens is removed.
- `most_frequent_token_sequence(V, I)` determines the most frequently occurring sequence of tokens (t_L, t_R) from V across all tokens in I .
- To speed up the determination of which pair of tokens from V occurs most frequently in D' , representing D' as an index I that maps tokens $t \in D'$ to their frequencies of occurrence in D can be utilized.
- An alternative approach is to employ a trie data structure. Upon each merge of two adjacent tokens, the vocabulary I or the index or trie representing it needs to be iteratively updated to reflect the change in token counts.

EXAMPLE: $D = \{ \text{“A HORSE, A HORSE, MY KINGDOM FOR A HORSE!”} \}$

$$R : \quad k \quad (t_L, t_R)$$

$$V = \{$$

$$I = \{$$

}

$$\}$$

EXAMPLE: $D = \{ \text{"A HORSE, A HORSE, MY KINGDOM FOR A HORSE!"} \}$

$$R: \frac{k}{(t_L, t_R)}$$

$$V = \{ , , !, _a, _f, _h, _k, _m, \\ A, d, e, f, g, h, i, \\ k, m, n, o, r, s, y, \\$$

$$I = \{ (A; 1), (_a; 2), (_f, o, r; 1), \\ (_h, o, r, s, e; 3), \\ (_k, i, n, g, d, o, m; 1), \\ (_m, y; 1), (,; 2), (!; 1) \}$$

}

$$D': ((A), (_h, o, r, s, e), (,), (_a), (_h, o, r, s, e), (,), (_m, y), (_k, i, n, g, d, o, m), (_f, o, r), (_a), (_h, o, r, s, e), (!))$$

EXAMPLE: $D = \{ \text{"A HORSE, A HORSE, MY KINGDOM FOR A HORSE!"} \}$

$$R: \frac{k \quad (t_L, t_R)}{1: \quad (o, r)}$$

$$V = \{ , , !, _a, _f, _h, _k, _m, \\ A, d, e, f, g, h, i, \\ k, m, n, o, r, s, y,$$

$$I = \{ (A; 1), (_a; 2), (_f, \text{or}; 1), \\ (_h, \text{or}, s, e; 3), \\ (_k, i, n, g, d, o, m; 1), \\ (_m, y; 1), (,; 2), (!; 1) \}$$

$\text{or}, \quad \}$

$$D': ((A), (_h, o, r, s, e), (,), (_a), (_h, o, r, s, e), (,), (_m, y), (_k, i, n, g, d, o, m), (_f, o, r), (_a), (_h, o, r, s, e), (!))$$

EXAMPLE: $D = \{ \text{"A HORSE, A HORSE, MY KINGDOM FOR A HORSE!"} \}$

$$\begin{array}{l}
 R: \frac{k \quad (t_L, t_R)}{1: \quad (o, r)} \\
 \quad 2: \quad (\textcolor{red}{_h}, \textcolor{red}{or}) \\
 \hline
 \end{array}
 \quad
 V = \{ , , !, _a, _f, _h, _k, _m, \\
 \quad A, d, e, f, g, h, i, \\
 \quad k, m, n, o, r, s, y, \\
 \quad \quad \textcolor{red}{_hor}, \\
 \quad \quad \quad or, \quad \}
 \quad
 I = \{ (A; 1), (_a; 2), (_f, or; 1), \\
 \quad (\textcolor{red}{_hor}, s, e; 3), \\
 \quad (_k, i, n, g, d, o, m; 1), \\
 \quad (_m, y; 1), (,; 2), (!; 1) \}$$

$$D': ((A), (_h, o, r, s, e), (,), (_a), (_h, o, r, s, e), (,), (_m, y), (_k, i, n, g, d, o, m), (_f, o, r), (_a), (_h, o, r, s, e), (!))$$

EXAMPLE: $D = \{ \text{"A HORSE, A HORSE, MY KINGDOM FOR A HORSE!"} \}$

$$\begin{array}{l}
 R: \begin{array}{c|c} k & (t_L, t_R) \\ \hline 1: & (o, r) \\ 2: & (_h, or) \\ 3: & (_hor, s) \end{array} \\
 \hline
 \end{array}
 \quad
 V = \{ , , !, _a, _f, _h, _k, _m, \\
 A, d, e, f, g, h, i, \\
 k, m, n, o, r, s, y, \\
 _hor, _hors, \\
 or, \}
 \quad
 I = \{ (A; 1), (_a; 2), (_f, or; 1), \\
 (_hors, e; 3), \\
 (_k, i, n, g, d, o, m; 1), \\
 (_m, y; 1), (, ; 2), (!; 1) \}$$

$$D': ((A), (_h, o, r, s, e), (,), (_a), (_h, o, r, s, e), (,), (_m, y), (_k, i, n, g, d, o, m), (_f, o, r), (_a), (_h, o, r, s, e), (!))$$

EXAMPLE: $D = \{ \text{"A HORSE, A HORSE, MY KINGDOM FOR A HORSE!"} \}$

$$R: \begin{array}{c|c} k & (t_L, t_R) \\ \hline 1: & (o, r) \\ 2: & (_h, or) \\ 3: & (_hor, s) \\ & \vdots \end{array}$$

$$V = \{ , , !, _a, _f, _h, _k, _m, \\ A, d, e, f, g, h, i, \\ k, m, n, o, r, s, y, \\ \text{dom, _for, _hor, _hors,} \\ _horse, _king, _my, or, \dots \}$$

$$I = \{ (A; 1), (_a; 2), (_for; 1), \\ (_horse; 3), \\ (_king, dom; 1), \\ (_my; 1), (,; 2), (!; 1) \}$$

$$D': ((A), (_h, o, r, s, e), (,), (_a), (_h, o, r, s, e), (,), (_m, y), (_k, i, n, g, d, o, m), (_f, o, r), (_a), (_h, o, r, s, e), (!))$$

EXAMPLE: $D = \{ \text{"A HORSE, A HORSE, MY KINGDOM FOR A HORSE!"} \}$

$R: \overline{k \quad (t_L, t_R)}$	$V = \{ , , !, _a, _f, _h, _k, _m,$	$I = \{ (A; 1), (_a; 2), (_for; 1),$
$1: (o, r)$	$A, d, e, f, g, h, i,$	$(_horse; 3),$
$2: (_h, or)$	$k, m, n, o, r, s, y,$	$(_king, dom; 1),$
$3: (_hor, s)$	$dom, _for, _hor, _hors,$	$(_my; 1), (, ; 2), (!; 1) \}$
$\vdots$	$_horse, _king, _my, or, \dots \}$	

$d': ((A), (_h, o, r, s, e), (,), (_a), (_h, o, r, s, e), (,), (_m, y), (_k, i, n, g, d, o, m), (_f, o, r), (_a), (_h, o, r, s, e), (!))$
 $1: ((A), (_h, or, s, e), (,), (_a), (_h, or, s, e), (,), (_m, y), (_k, i, n, g, d, o, m), (_f, or), (_a), (_h, or, s, e), (!))$
 $2: ((A), (_hor, s, e), (,), (_a), (_hor, s, e), (,), (_m, y), (_k, i, n, g, d, o, m), (_f, or), (_a), (_hor, s, e), (!))$
 $3: ((A), (_hors, e), (,), (_a), (_hors, e), (,), (_m, y), (_k, i, n, g, d, o, m), (_f, or), (_a), (_hors, e), (!))$
 $\vdots$
 $k: ((A), (_horse), (,), (_a), (_horse), (,), (_my), (_king), (dom), (_for), (_a), (_horse), (!))$

QUESTIONS?

SUBWORD TOKENISATION

Properties:

- Bottom-up analysis of texts.
- Naïve heuristic to identify (frequent) morphemes.
- Results in partly human-interpretable tokens, partly non-interpretable ones.
- Language-specific knowledge and resources may be required: pre-tokenization heuristic.
- The vocabulary of tokens can be bounded.
- Token normalization is optional and depends on the application.

Applicability:

- Computing document embeddings in neural networks with (GPU) memory limitations.

REMARKS

- If the tokenizer is used on a document which contains character that do not occur in alphabet Σ , no tokens can be formed: This is why a special token for unknown (out-of-vocabulary, OOV) tokens, [unk] is often introduced as a catch-all token for such and similar cases.
- A variant of Byte-Pair Encoding (BPE) is used by GPT-2: byte-level BPE. It uses all 256 different bytes as basis vocabulary to avoid the [unk] token completely.
- BERT and many of its variants use WordPiece, which is an extension of BPE that uses a different function to find the most likely merge, instead of the most frequent adjacent tokens. This avoids merging subwords that also often appear independently.
- The tokenizers Unigram [Kudo, 2018] and SentencePiece [Kudo and Richardson, 2018] work in reverse to WordPiece: they add all possible tokens to the vocabulary, then iteratively remove tokens until the desired vocabulary size is reached.
- Providers of closed-source language models often share their tokenizer implementation open source, since customer need to be able to reliably calculate the number of tokens they may process in order to reliably estimate usage costs. [OpenAI tiktoken]

TOKEN REMOVAL

Undesired tokens (so-called stop words) are removed (process called stopping) to reduce data size, sparsity, and to improve performance on downstream tasks.

- Frequent tokens (per corpus): E.g., remove one occurrence of Wikipedia from each Wikipedia article.
- Function word tokens (per language): E.g., the, of, and, etc.; but what happens to a phrase like to be or not to be ?
- Punctuation-only tokens: This would remove smiley like ‘;-)’, which are meaningful symbols.
- Number-only tokens
- Short tokens: xp, ma, pm, ben e king, el paso, master p, gm, j lo, ...

TOKEN REMOVAL

Source text: (34 tokens)

The idea of giving computers the ability to process human language is as old as the idea of computers themselves. This book is about the implementation and implications of that exciting idea.

Stopped text: (16 tokens)

The idea of giving computers the ability to process human language is as old as the idea of computers themselves. This book is about the implementation and implications of that exciting idea.

QUESTIONS?